Lecture 5/Module 2 & 3
CS3212 Algorithms

Aya Zirikly
09/08/2026

QuickSort

Invention: Tony Hoare (1959).

Paradigm: Divide-and-Conquer (Heavy Divide, Trivial Combine).

Steps:

1. **Divide:** Select an element as pivot p. Partition array A[low..high] into two sub-arrays such that all elements in Left <= p and all elements in Right >= p.
2. **Conquer:** Recursively sort Left and Right sub-arrays.
3. **Combine:** Do nothing! Elements are sorted in-place.

Key Contrast with Merge Sort:

QuickSort

Invention: Tony Hoare (1959).

Paradigm: Divide-and-Conquer (Heavy Divide, Trivial Combine).

Steps:

1. **Divide:** Select an element as pivot p. Partition array A[low..high] into two sub-arrays such that all elements in Left <= p and all elements in Right >= p.
2. **Conquer:** Recursively sort Left and Right sub-arrays.
3. **Combine:** Do nothing! Elements are sorted in-place.

Key Contrast with Merge Sort: Merge Sort does light work during divide (splitting at mid) and heavy work during combine (merging). QuickSort does heavy work during divide (partitioning) and zero work during combine.

Input Array: [6, 2, 8, 3, 1, 5, 4]

Partitioning Scheme: Lomuto Partitioning (Pivot = Last Element)

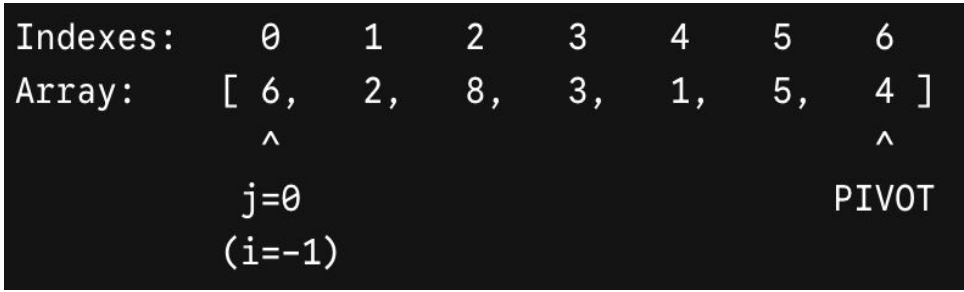
Part 1: Detailed Mechanics of Partitioning (First Call)

Initial State:

- Array: [6, 2, 8, 3, 1, 5, 4]
- Pivot value: **4** (index 6)
- Pointer i = -1 (tracks boundary of elements <= pivot)
- Pointer j = 0 (scans array from left to right)

Step 1 (j = 0, value = 6):

- 6 > 4 (Greater than pivot) → Do nothing. i = -1.



```
[6, 2, 8, 3, 1, 5, 4]
```

Step 2 (j = 1, value = 2):

- 2 <= 4 (Less than or equal to pivot) → Increment i to 0, Swap A[i] and A[j] (Swap 6 and 2).
- Array state: [2, 6, 8, 3, 1, 5, 4]

```
Indexes:    0    1    2    3    4    5    6
Array:    [ 2,   6,   8,   3,   1,   5,   4 ]
            ^    ^                        ^
          i=0  j=1                     PIVOT
```

Step 3 (j = 2, value = 8):

- 8 > 4 → Do nothing. i = 0.

Step 4 (j = 3, value = 3):

- 3 <= 4 → Increment i to 1, Swap A[i] and A[j] (Swap 6 and 3).
- Array state: [2, 3, 8, 6, 1, 5, 4]

```
Indexes:    0    1    2    3    4    5    6
Array:    [ 2,   3,   8,   6,   1,   5,   4 ]
               ^         ^                  ^
             i=1       j=3               PIVOT
```

Step 5 (j = 4, value = 1):

- 1 <= 4 → Increment i to 2, Swap A[i] and A[j] (Swap 8 and 1).
- Array state: [2, 3, 1, 6, 8, 5, 4]

```
Indexes:    0    1    2    3    4    5    6
Array:    [ 2,   3,   1,   6,   8,   5,   4 ]
                       ^         ^         ^
                      i=2       j=4      PIVOT
```

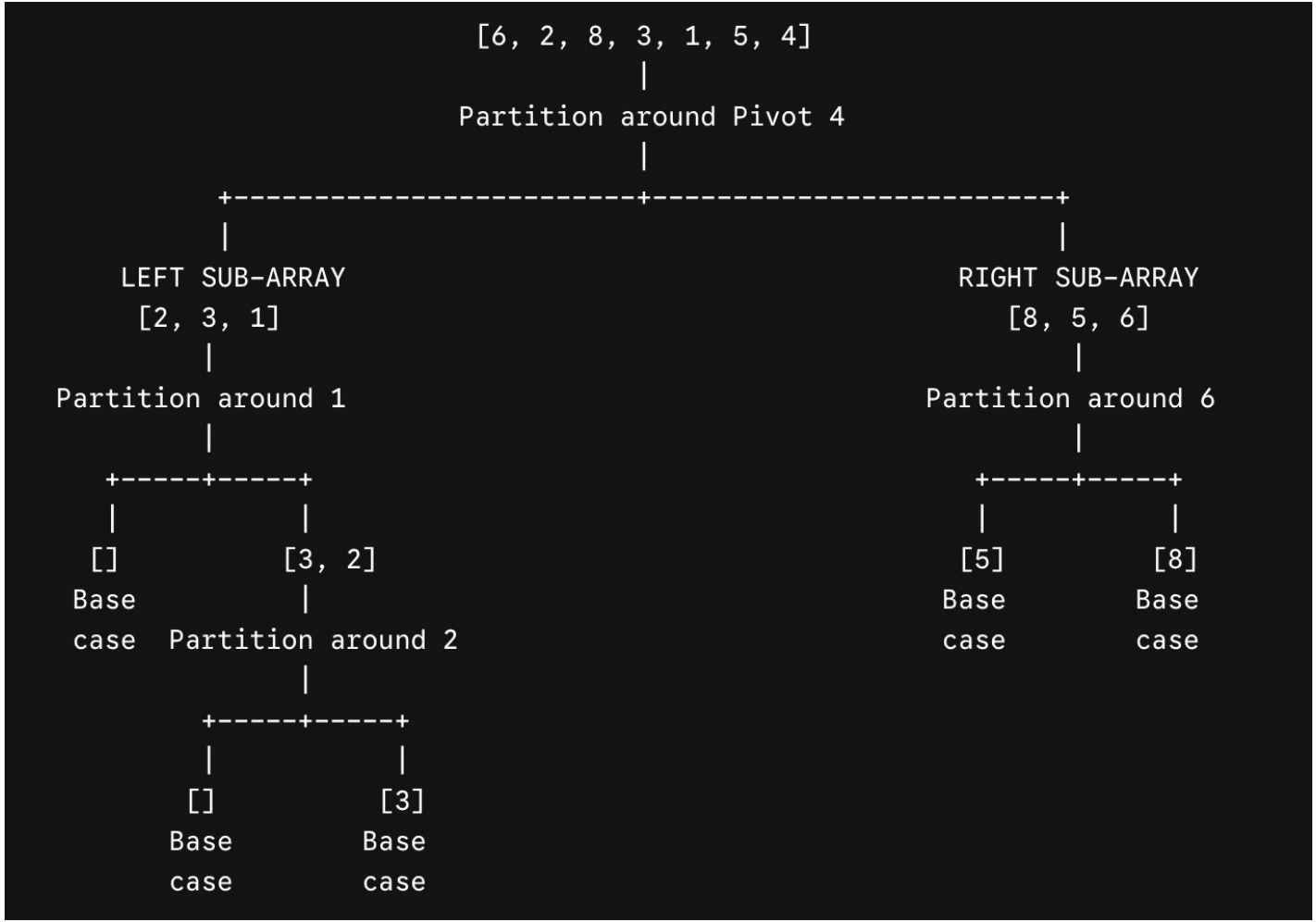
Step 6 (j = 5, value = 5):

- 5 > 4 → Do nothing. i = 2.

Final Pivot Placement:

- Swap Pivot A[high] with A[i + 1] (Swap 4 and 6 at index 3).
- Array state: [2, 3, 1, 4, 8, 5, 6]

```
Partition Complete:
Left Partition (< 4)   | PIVOT | Right Partition (> 4)
-----------------------+-------+----------------------
  [ 2,   3,   1 ]      | [4]   |   [ 8,   5,   6 ]
```


[6, 2, 8, 3, 1, 5, 4]
Partition around Pivot 4
LEFT SUB-ARRAY
[2, 3, 1]
RIGHT SUB-ARRAY
[8, 5, 6]
Partition around 1
Partition around 6
[]
Base case
[3, 2]
[5]
Base case
[8]
Base case
Partition around 2
[]
Base case
[3]
Base case

Recursion Execution

Root Call: QuickSort([6, 2, 8, 3, 1, 5, 4])

- Pivot = 4 → Partition result: [2, 3, 1] | **[4]** | [8, 5, 6]
- **[4] is locked in place.**

Left Branch Call 1: QuickSort([2, 3, 1])

- Pivot = 1
- Elements 2 and 3 are > 1. Swap 1 into index 0.
- Partition result: [] | **[1]** | [3, 2]
- **[1] is locked in place.**

Left Branch Call 2 (Sub-Right): QuickSort([3, 2])

- Pivot = 2
- Element 3 > 2. Swap 2 with 3.
- Partition result: [] | **[2]** | [3]
- **[2] and [3] are locked in place.**

Right Branch Call 1: QuickSort([8, 5, 6])

- Pivot = 6
- 5 <= 6 (stays left), 8 > 6 (moves right).
- Partition result: [5] | **[6]** | [8]
- **[5], [6], and [8] are locked in place.**

```
[1] -> [2] -> [3] -> [4] -> [5] -> [6] -> [8]

Final Sorted Output: [1, 2, 3, 4, 5, 6, 8]
```

QuickSort Best-Case Analysis

Best Case Scenario: Pivot consistently splits array into two equal halves of size `n/2`.

Recurrence: `T(n) = 2 * T(n/2) + c * n`

Recursion Tree:

- Root cost: `c * n`
- Level 1: 2 nodes of size `n/2` -> sum = `c * n`
- Level `i`: `2^i` nodes of size `n / 2^i` -> sum = `c * n`
- Tree Height: `log_2 n`

Complexity:

- Time: `Theta(n log n)`
- Space: `O(log n)`

QuickSort Best-Case Analysis

Best Case Scenario: Pivot consistently splits array into two equal halves of size `n/2`.

Recurrence: `T(n) = 2 * T(n/2) + c * n`

Recursion Tree:

- Root cost: `c * n`
- Level 1: 2 nodes of size `n/2` -> sum = `c * n`
- Level `i`: `2^i` nodes of size `n / 2^i` -> sum = `c * n`
- Tree Height: `log_2 n`

Complexity:

- Time: `Theta(n log n)`
- Space: `O(log n)`

Stack of memory needed to remember where to return after a recursive function finishes

QuickSort Worst-Case Analysis

Worst Case Scenario: Input is already sorted (or reverse sorted) and pivot is always selected as the last/first element.

Splits: Partition splits size `n` into `0` elements and `n - 1` elements.

Recurrence: `T(n) = T(n - 1) + c * n`

- Level 0: Cost `c * n`
- Level 1: Cost `c * (n - 1)`
- Level 2: Cost `c * (n - 2)`
- Total Sum: `c * (n + (n - 1) + (n - 2) + ... + 1) = c * n * (n + 1) / 2`

Complexity:

- Time: `Theta(n^2)`
- Space: `O(n)`

QuickSort Worst-Case Analysis

1. **Pivot Selection:** Suppose you choose the last element as the pivot, and that element happens to be the smallest value in the current array (e.g., trying to sort an already reverse-sorted array [5, 4, 3, 2, 1]).
2. **Partitioning:** The algorithm compares all remaining n - 1 elements to the pivot (1).
    - **Elements smaller than pivot:** 0 elements
    - **Elements larger than pivot:** n - 1 elements ([5, 4, 3, 2])
3. **Subproblem Division:**
    - Left side gets a subarray of size **0**
    - The pivot takes its fixed place in the array
    - Right side gets a subarray of size **n - 1**

QuickSort Average-Case Analysis

Scenario: Pivot creates a skewed split (e.g., 90% right, 10% left at every step).

Recurrence: `T(n) = T(n/10) + T(9n/10) + c * n`

Recursion Tree Analysis:

- Work per full level: `c * n`
- Shortest branch to leaf: `log_10 n`
- Longest branch to leaf: `log_(10/9) n = 2.18 log_2 n`
- Sum across levels: Upper bounded by `c * n * (2.18 log_2 n)`

Takeaway: Even highly unbalanced splits (9-to-1 or 99-to-1) yield `Theta(n log n)` time! Skewed split ratios only change the constant base of the logarithm.

Tree Height Under a 90/10 Split

If the array shrinks by taking 90% of the remaining elements at each step, the longest branch shrinks as:

$n \rightarrow 0.9n \rightarrow (0.9)^2 * n \rightarrow ... \rightarrow 1$

To find how many steps h it takes for $(0.9)^h * n = 1$:

$h = \log_{(1/0.9)}(n) \approx \log_{1.11}(n) \approx 22.5 * \log_2(n)$

Summary

	Time complexity	Space complexity	
Best case	Θ(n log n)	O(log n)	Pivot always splits array into equal halves (n/2 and n/2)
Average case	Θ(n log n)	O(log n)	Pivot provides reasonably balanced splits over random inputs
Worst case	$O(n^2)$	O(n) or O(log n)*	Pivot is always extreme value, yielding 0 and n-1 splits

*Worst-case space reduces to O(log n) if using tail-recursion optimization on the smaller partition.

QuickSelect

Problem: Find the k-th smallest element in an unsorted array of size `n` (e.g., finding the median).

Core Idea: Partition like QuickSort, but recurse into ONLY the single partition containing index `k`.

Recurrence (Average Case): `T(n) = T(n/2) + c * n`

Recursion Tree Analysis:

- Single-path tree (no branching factor!).
- Level 0 work: `c * n`
- Level 1 work: `c * n / 2`
- Level 2 work: `c * n / 4`
- Total Work: `c * n * (1 + 1/2 + 1/4 + 1/8 + ...) <= 2 * c * n`

QuickSelect

Problem: Find the k-th smallest element in an unsorted array of size `n` (e.g., finding the median).

Core Idea: Partition like QuickSort, but recurse into ONLY the single partition containing index `k`.

Recurrence (Average Case): `T(n) = T(n/2) + c * n`

Recursion Tree Analysis:

- Single-path tree (no branching factor!).
- Level 0 work: `c * n`
- Level 1 work: `c * n / 2`
- Level 2 work: `c * n / 4`
- Total Work: `c * n * (1 + 1/2 + 1/4 + 1/8 + ...) <= 2 * c * n`
-

Complexity

- Average Time: `Theta(n)`
- Worst Case Time: `Theta(n^2)` (if bad pivots chosen repeatedly)

QuickSelect worst case

In the **worst case**, bad pivots are chosen every single time (for example, the pivot is always the smallest element, splitting the array into 0 elements on one side and n-1 elements on the other).

Instead of dropping half the array, you only eliminate **1 single element** per step:

- Step 1: Look at n elements.
- Step 2: Look at n - 1 elements.
- Step 3: Look at n - 2 elements.
- ...
- Last step: Look at 1 element.

$$n + (n - 1) + (n - 2) + ... + 2 + 1$$

$$[n * (n + 1)] / 2 = (n^2 + n) / 2$$

3-Way Merge Sort

Algorithm: Split array into 3 equal parts of size `n/3`, recursively sort each, and merge 3 sorted arrays using a 3-pointer comparison.

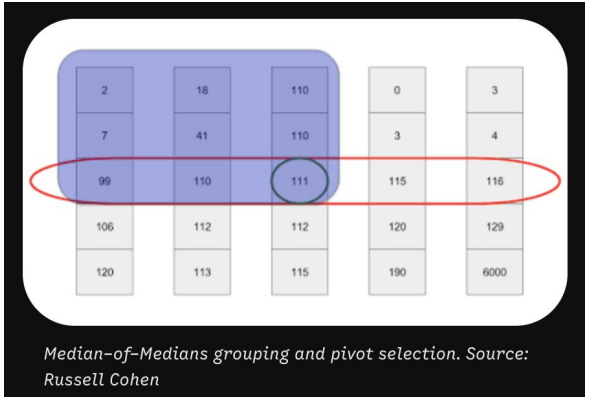
Recurrence: `T(n) = 3 * T(n/3) + c * n`

Recursion Tree Mechanics:

- Branching factor: `a = 3`
- Problem size reduction: `b = 3`
- Level `i` work: `3^i * c * (n / 3^i) = c * n`
- Tree Height: `h = log_3 n`

Total Work: `c * n * log_3 n = (c / log_2 3) * n log_2 n = Theta(n log n)`
Takeaway: Increasing the divide factor `b` reduces tree height, but increases combine work proportionally, keeping overall complexity at `Theta(n log n)`.

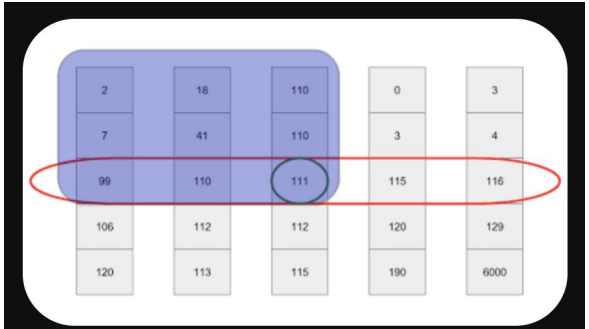
Deterministic Linear-Time Selection (Median-of-Medians) BFPRT



Median-of-Medians grouping and pivot selection. Source: Russell Cohen

- "Divide n elements into groups of 5" Break the big array into tiny blocks of 5 elements each. If you have 100 elements, you get 20 groups of 5.
- "Find median of each group of 5 (takes O(1) per group -> O(n) total)" Finding the middle element of just 5 numbers takes a fixed, tiny amount of time—a constant O(1) operation. Doing this across all n/5 groups takes linear time, O(n).
- "Recursively find median of the n/5 medians -> Cost = T(n/5)" Now you collect all those group medians (there are n/5 of them). To find the middle value of that collection, you run this exact same algorithm on those n/5 items. That sub-problem takes T(n/5) time.
- "Use this median-of-medians as pivot to partition array" You take the resulting number and use it as your QuickSelect pivot.
- Guaranteed `Theta(n)` worst-case time!

Why BFPRT



Median-of-Medians grouping and pivot selection. Source: Russell Cohen

Eliminates the worst-case vulnerability of standard QuickSelect, guaranteeing worst-case performance in **O(n)** time.

Comparison-Based Sorting Model

- **Definition:** Sorting algorithms that only access input elements through direct pairwise comparisons (`A[i] <= A[j]`, `A[i] > A[j]`).
- **Algorithms in this Model:** Merge Sort, QuickSort, HeapSort, Insertion Sort, Bubble Sort.
- **Excluded Algorithms:** Counting Sort, Radix Sort, Bucket Sort (these examine digit/bit structure directly, not pairwise comparisons).
- **Fundamental Question:** What is the theoretical minimum number of comparisons needed to sort n arbitrary items?

The Decision Tree Model

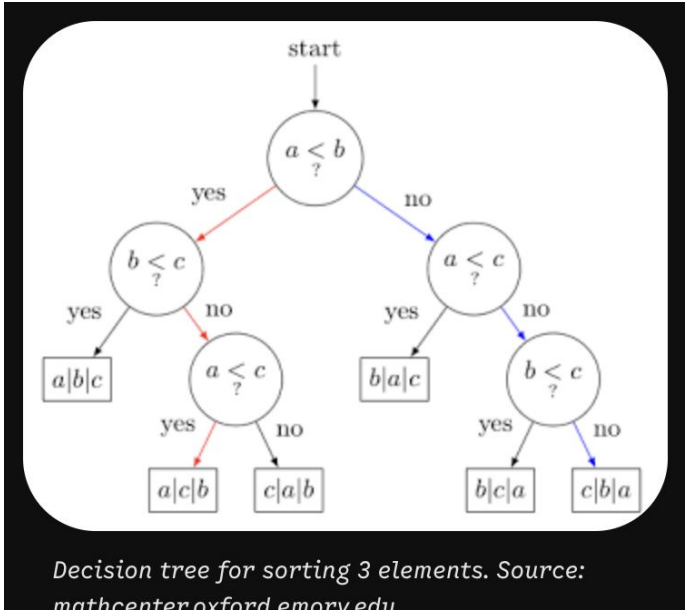
Concept: Model the execution of any comparison sort as a full binary decision tree.

Nodes:

- Internal Nodes: Represent comparisons `i : j` (comparing element at index `i` with index `j`).
- Edges: Represent outcome (`<=` goes left, `>` goes right).
- Leaf Nodes: Represent a final sorted permutation of input `<a_1, a_2, ..., a_n>`

Decision tree for sorting 3 elements `<a1, a2, a3>`.

- Root compares `a1:a2`.
- Left branch compares `a2:a3`, right branch compares `a1:a3`.
- Total leaves = `3!= 6` possible outcomes.



Decision tree for sorting 3 elements. Source: mathcenter.oxford.emory.edu

Properties of the Sorting Decision Tree

Number of Leaves (L):

- An array of size `n` has `n!` possible permutations (`n * (n - 1) * ... * 1`).
- To correctly sort every possible input, the decision tree must contain at least one leaf for each permutation.
- Therefore: `L >= n!`

Leaf Nodes (L): Each leaf node in the decision tree represents a final, specific sorted permutation. If L is less than n!, there would be at least one possible input ordering that the algorithm could never produce, meaning it would fail to sort that input.

Tree Height (h):

- Height `h` equals the maxi t to leaf.
- Represents the **worst-ca**

Binary Tree Capacity:

- A binary tree of height `h` has at most `2^h` leaves.
- Therefore: `2^h >= L >= n!`

Properties of the Sorting Decision Tree

Number of Leaves (L):

- An array of size `n` has `n!` possible permutations (`n * (n - 1) * ... * 1`).
- To correctly sort every possible input, the decision tree must contain at least one leaf for each permutation.
- Therefore: `L >= n!`

Tree Height (h):

- Height `h` equals the maximum number of comparisons performed on the longest path from root to leaf.
- Represents the **worst-case running time** of the sorting algorithm.

Binary Tree Capacity:

- A binary tree of height `h` has at most `2^h` leaves.
- Therefore: `2^h >= L >= n!`

Mathematical Derivation of the Lower Bound

Starting Inequality: `2^h >= n!`

Taking Base-2 Logarithm of Both Sides: `h >= log_2 (n!)`

Expanding `log_2 (n!)`: `log_2 (n!) = log_2 (1 * 2 * 3 * ... * n)`

`log_2 (n!) = log_2(1) + log_2(2) + ... + log_2(n)`

To find a lower bound, we can safely make the expression **smaller** by dropping the first half of the terms (from `1` to `n/2 - 1`):

`log_2(n!) >= log_2(n/2) + log_2(n/2 + 1) + ... + log_2(n)`

- **Why is this valid?** Dropping positive numbers from a sum only makes the total smaller.
- **How many terms are left?** Exactly `n/2` terms remain in the second half.

Mathematical Derivation of the Lower Bound

Replace Every Remaining Term with the Smallest One

Look at the terms left in the sum: `log_2(n/2)`, `log_2(n/2 + 1)`, ..., `log_2(n)`.

The smallest term in this remaining group is the very first one: `log_2(n/2)`.

If we replace **every** term with this smallest value, the sum gets even smaller or equal:

`log_2(n!) >= log_2(n/2) + log_2(n/2) + ... + log_2(n/2)` *(repeated n/2 times)*

Mathematical Derivation of the Lower Bound

Simplify the Logarithm

Using log rules again (`log(a / b) = log(a) - log(b)`):

`log_2(n/2) = log_2(n) - log_2(2) = log_2(n) - 1`

Do the Multiplication

Now we simply multiply the number of terms (`n/2`) by the value of each term (`log_2(n) - 1`):

`log_2(n!) >= (n/2) * (log_2(n) - 1)`

Distribute the `(n/2)` through the parentheses:

`log_2(n!) >= (n/2) * log_2(n) - (n/2)`

Stirling's Approximation & Final Proof

Stirling's Formula for Factorials: `n! approx = sqrt(2 * pi * n) * (n / e)^n`

log_2(n!) = log_2((n / e)^n) + log_2(sqrt(2 * pi * n))

log_2((n / e)^n) = n * log_2(n / e) = n log_2 n - n log_2 e

log_2(sqrt(2 * pi * n)) = (1/2) log_2(2 * pi * n) = (1/2) log_2(2 * pi) + (1/2) log_2 n

log_2(n!) = n log_2 n - n log_2 e + O(log n)

Stirling's Approximation & Final Proof

When analyzing growth as n approaches infinity, we look at which term grows the fastest:

- n log_2 n grows extremely fast.
- n log_2 e (approx 1.44n) grows linearly.
- O(log n) grows very slowly.

Because n log_2 n completely dominates both n and log n for large n, all lower-order terms can be dropped when expressing the asymptotic bound:

log_2(n!) = Theta(n log n)

Algorithm	Best-Case Time	Average-Case Time	Worst-Case Time	Auxiliary Space	Stable?	Paradigm
Merge Sort	Theta(n log n)	Theta(n log n)	Theta(n log n)	O(n)	Yes	Divide & Conquer
QuickSort	Theta(n log n)	Theta(n log n)	Theta(n^2)	O(log n)	No	Divide & Conquer
HeapSort	Theta(n log n)	Theta(n log n)	Theta(n log n)	O(1)	No	Selection / Heap
QuickSelect	Theta(n)	Theta(n)	Theta(n^2)	O(log n)	No	Prune & Conquer
Median-of-Medians	Theta(n)	Theta(n)	Theta(n)	O(log n)	No	Divide & Conquer